JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio

Class CharBuffer

java.lang.Object
    java.nio.Buffer
        java.nio.CharBuffer

All Implemented Interfaces:

`Appendable`, `CharSequence`, `Comparable<CharBuffer>`, `Readable`

```
public abstract class CharBuffer
extends Buffer
implements Comparable<CharBuffer>, Appendable, CharSequence, Readable
```

A char buffer.

This class defines four categories of operations upon char buffers:

- Absolute and relative *`get`* and *`put`* methods that read and write single chars;
- Relative *`bulk get`* methods that transfer contiguous sequences of chars from this buffer into an array; and
- Relative *`bulk put`* methods that transfer contiguous sequences of chars from a char array, a string, or some other char buffer into this buffer; and
- Methods for `compacting`, `duplicating`, and `slicing` a char buffer.

Char buffers can be created either by *`allocation`*, which allocates space for the buffer's content, by *`wrapping`* an existing char array or string into a buffer, or by creating a *view* of an existing byte buffer.

Like a byte buffer, a char buffer is either *direct* or *non-direct*. A char buffer created via the `wrap` methods of this class will be non-direct. A char buffer created as a view of a byte buffer will be direct if, and only if, the byte buffer itself is direct. Whether or not a char buffer is direct may be determined by invoking the `isDirect` method.

This class implements the `CharSequence` interface so that character buffers may be used wherever character sequences are accepted, for example in the regular-expression package `java.util.regex`.

Methods in this class that do not otherwise have a value to return are specified to return the buffer upon which they are invoked. This allows method invocations to be chained. The sequence of statements

```
cb.put("text/");
cb.put(subtype);
cb.put("; charset=");
cb.put(enc);
```

can, for example, be replaced by the single statement

```
cb.put("text/").put(subtype).put("; charset=").put(enc);
```

Since:

1.4

Method Summary

Instance Methods | **Abstract Methods** | **Concrete Methods**

Modifier and Type	Method	Description
static `CharBuffer`	**`allocate`**`(int capacity)`	Allocates a new char buffer.
`CharBuffer`	**`append`**`(char c)`	Appends the specified char to this buffer *(optional operation)*.
`CharBuffer`	**`append`**`(CharSequence csq)`	Appends the specified character sequence to this buffer *(optional operation)*.
`CharBuffer`	**`append`**`(CharSequence csq, int start, int end)`	Appends a subsequence of the specified character sequence to this buffer *(optional operation)*.
`char[]`	**`array`**`()`	

		Returns the char array that backs this buffer *(optional operation)*.
int	**arrayOffset**()	Returns the offset within this buffer's backing array of the first element of the buffer *(optional operation)*.
abstract **CharBuffer**	**asReadOnlyBuffer**()	Creates a new, read-only char buffer that shares this buffer's content.
char	**charAt**(int index)	Reads the character at the given index relative to the current position.
IntStream	**chars**()	Returns a stream of `int` zero-extending the `char` values from this sequence.
abstract **CharBuffer**	**compact**()	Compacts this buffer *(optional operation)*.
int	**compareTo**(**CharBuffer** that)	Compares this buffer to another.
abstract **CharBuffer**	**duplicate**()	Creates a new char buffer that shares this buffer's content.
boolean	**equals**(**Object** ob)	Tells whether or not this buffer is equal to another object.

abstract char	**get**()	Relative *get* method.
CharBuffer	**get**(char[] dst)	Relative bulk *get* method.
CharBuffer	**get**(char[] dst, int offset, int length)	Relative bulk *get* method.
abstract char	**get**(int index)	Absolute *get* method.
boolean	**hasArray**()	Tells whether or not this buffer is backed by an accessible char array.
int	**hashCode**()	Returns the current hash code of this buffer.
abstract boolean	**isDirect**()	Tells whether or not this char buffer is direct.
int	**length**()	Returns the length of this character buffer.
abstract **ByteOrder**	**order**()	Retrieves this buffer's byte order.
abstract **CharBuffer**	**put**(char c)	Relative *put* method *(optional operation)*.
CharBuffer	**put**(char[] src)	Relative bulk *put* method *(optional operation)*.
CharBuffer	**put**(char[] src, int offset, int length)	Relative bulk *put* method *(optional operation)*.
CharBuffer	**put**(**CharBuffer** src)	Relative bulk *put* method *(optional operation)*.
abstract **CharBuffer**	**put**(int index, char c)	

		Absolute *put* method *(optional operation)*.
CharBuffer	**put**(**String** src)	Relative bulk *put* method *(optional operation)*.
CharBuffer	**put**(**String** src, int start, int end)	Relative bulk *put* method *(optional operation)*.
int	**read**(**CharBuffer** target)	Attempts to read characters into the specified character buffer.
abstract **CharBuffer**	**slice**()	Creates a new char buffer whose content is a shared subsequence of this buffer's content.
abstract **CharBuffer**	**subSequence**(int start, int end)	Creates a new character buffer that represents the specified subsequence of this buffer, relative to the current position.
String	**toString**()	Returns a string containing the characters in this buffer.
static **CharBuffer**	**wrap**(char[] array)	Wraps a char array into a buffer.
static **CharBuffer**	**wrap**(char[] array, int offset, int length)	Wraps a char array into a buffer.
static **CharBuffer**	**wrap**(**CharSequence** csq)	Wraps a character sequence into a buffer.

static CharBuffer	wrap(CharSequence csq, int start, int end)	Wraps a character sequence into a buffer.

Methods inherited from class java.nio.Buffer

capacity, clear, flip, hasRemaining, isReadOnly, limit, limit, mark, position, position, remaining, reset, rewind

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.lang.CharSequence

codePoints

Method Detail

allocate

```
public static CharBuffer allocate(int capacity)
```

Allocates a new char buffer.

The new buffer's position will be zero, its limit will be its capacity, its mark will be undefined, and each of its elements will be initialized to zero. It will have a `backing array`, and its `array offset` will be zero.

Parameters:

`capacity - The new buffer's capacity, in chars`

Returns:

`The new char buffer`

Throws:

`IllegalArgumentException - If the capacity is a negative integer`

wrap

```
public static CharBuffer wrap(char[] array,
                              int offset,
                              int length)
```

Wraps a char array into a buffer.

The new buffer will be backed by the given char array; that is, modifications to the buffer will cause the array to be modified and vice versa. The new buffer's capacity will be `array.length`, its position will be `offset`, its limit will be `offset + length`, and its mark will be undefined. Its `backing array` will be the given array, and its `array offset` will be zero.

Parameters:

`array - The array that will back the new buffer`

`offset - The offset of the subarray to be used; must be non-negative and no larger than array.length. The new buffer's position will be set to this value.`

`length - The length of the subarray to be used; must be non-negative and no larger than array.length - offset. The new buffer's limit will be set to offset + length.`

Returns:

`The new char buffer`

Throws:

`IndexOutOfBoundsException - If the preconditions on the offset and length parameters do not hold`

wrap

```
public static CharBuffer wrap(char[] array)
```

Wraps a char array into a buffer.

The new buffer will be backed by the given char array; that is, modifications to the buffer will cause the array to be modified and vice versa. The new buffer's capacity and limit will be `array.length`, its position will be zero, and its mark will be undefined. Its `backing array` will be the given array, and its `array offset>` will be zero.

Parameters:

`array - The array that will back this buffer`

Returns:

`The new char buffer`

read

```
public int read(CharBuffer target)
         throws IOException
```

Attempts to read characters into the specified character buffer. The buffer is used as a repository of characters as-is: the only changes made are the

results of a put operation. No flipping or rewinding of the buffer is performed.

Specified by:

`read` in interface `Readable`

Parameters:

`target - the buffer to read characters into`

Returns:

`The number of characters added to the buffer, or -1 if this source of characters is at its end`

Throws:

`IOException - if an I/O error occurs`

`NullPointerException - if target is null`

`ReadOnlyBufferException - if target is a read only buffer`

Since:

`1.5`

wrap

```
public static CharBuffer wrap(CharSequence csq,
                              int start,
                              int end)
```

Wraps a character sequence into a buffer.

The content of the new, read-only buffer will be the content of the given character sequence. The buffer's capacity will be `csq.length()`, its position will be `start`, its limit will be `end`, and its mark will be undefined.

Parameters:

`csq - The character sequence from which the new character buffer is to be created`

`start - The index of the first character to be used; must be non-negative and no larger than csq.length(). The new buffer's position will be set to this value.`

`end - The index of the character following the last character to be used; must be no smaller than start and no larger than csq.length(). The new buffer's limit will be set to this value.`

Returns:

`The new character buffer`

Throws:

`IndexOutOfBoundsException - If the preconditions on the start and end parameters do not hold`

wrap

```
public static CharBuffer wrap(CharSequence csq)
```

Wraps a character sequence into a buffer.

The content of the new, read-only buffer will be the content of the given character sequence. The new buffer's capacity and limit will be `csq.length()`, its position will be zero, and its mark will be undefined.

Parameters:

`csq - The character sequence from which the new character buffer is to be created`

Returns:

`The new character buffer`

slice

```
public abstract CharBuffer slice()
```

Creates a new char buffer whose content is a shared subsequence of this buffer's content.

The content of the new buffer will start at this buffer's current position. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's position will be zero, its capacity and its limit will be the number of chars remaining in this buffer, and its mark will be undefined. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Returns:

The new char buffer

duplicate

```
public abstract CharBuffer duplicate()
```

Creates a new char buffer that shares this buffer's content.

The content of the new buffer will be that of this buffer. Changes to this buffer's content will be visible in the new buffer, and vice versa; the two buffers' position, limit, and mark values will be independent.

The new buffer's capacity, limit, position, and mark values will be identical to those of this buffer. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Returns:

The new char buffer

asReadOnlyBuffer

```
public abstract CharBuffer asReadOnlyBuffer()
```

Creates a new, read-only char buffer that shares this buffer's content.

The content of the new buffer will be that of this buffer. Changes to this buffer's content will be visible in the new buffer; the new buffer itself, however, will be read-only and will not allow the shared content to be modified. The two buffers' position, limit, and mark values will be independent.

The new buffer's capacity, limit, position, and mark values will be identical to those of this buffer.

If this buffer is itself read-only then this method behaves in exactly the same way as the `duplicate` method.

Returns:

`The new, read-only char buffer`

get

```
public abstract char get()
```

Relative *get* method. Reads the char at this buffer's current position, and then increments the position.

Returns:

`The char at the buffer's current position`

Throws:

`BufferUnderflowException - If the buffer's current position is not smaller than its limit`

put

```
public abstract CharBuffer put(char c)
```

Relative *put* method *(optional operation)*.

Writes the given char into this buffer at the current position, and then increments the position.

Parameters:

`c - The char to be written`

Returns:

`This buffer`

Throws:

`BufferOverflowException - If this buffer's current position is not smaller than its limit`

`ReadOnlyBufferException - If this buffer is read-only`

get

```
public abstract char get(int index)
```

Absolute *get* method. Reads the char at the given index.

Parameters:

`index - The index from which the char will be read`

Returns:

`The char at the given index`

Throws:

`IndexOutOfBoundsException - If index is negative or not smaller than the buffer's limit`

put

```
public abstract CharBuffer put(int index,
                               char c)
```

Absolute *put* method *(optional operation)*.

Writes the given char into this buffer at the given index.

Parameters:

`index - The index at which the char will be written`

`c - The char value to be written`

Returns:

`This buffer`

Throws:

`IndexOutOfBoundsException - If index is negative or not smaller than the buffer's limit`

`ReadOnlyBufferException - If this buffer is read-only`

get

```
public CharBuffer get(char[] dst,
                      int offset,
                      int length)
```

Relative bulk *get* method.

This method transfers chars from this buffer into the given destination array. If there are fewer chars remaining in the buffer than are required to satisfy the request, that is, if `length > remaining()`, then no chars are transferred and a `BufferUnderflowException` is thrown.

Otherwise, this method copies `length` chars from this buffer into the given array, starting at the current position of this buffer and at the given offset in the array. The position of this buffer is then incremented by `length`.

In other words, an invocation of this method of the form `src.get(dst, off, len)` has exactly the same effect as the loop

```
for (int i = off; i < off + len; i++)
    dst[i] = src.get():
```

except that it first checks that there are sufficient chars in this buffer and it is potentially much more efficient.

Parameters:

`dst` - The array into which chars are to be written

`offset` - The offset within the array of the first char to be written; must be non-negative and no larger than `dst.length`

`length` - The maximum number of chars to be written to the given array; must be non-negative and no larger than `dst.length - offset`

Returns:

This buffer

Throws:

`BufferUnderflowException` - If there are fewer than length chars remaining in this buffer

`IndexOutOfBoundsException` - If the preconditions on the offset and length parameters do not hold

get

`public CharBuffer get(char[] dst)`

Relative bulk *get* method.

This method transfers chars from this buffer into the given destination array. An invocation of this method of the form `src.get(a)` behaves in exactly the same way as the invocation

```
src.get(a, 0, a.length)
```

Parameters:

`dst` - The destination array

Returns:

This buffer

Throws:

`BufferUnderflowException` - If there are fewer than length chars remaining in this buffer

put

```
public CharBuffer put(CharBuffer src)
```

Relative bulk *put* method *(optional operation)*.

This method transfers the chars remaining in the given source buffer into this buffer. If there are more chars remaining in the source buffer than in this buffer, that is, if `src.remaining() > remaining()`, then no chars are transferred and a `BufferOverflowException` is thrown.

Otherwise, this method copies *n* = `src.remaining()` chars from the given buffer into this buffer, starting at each buffer's current position. The positions of both buffers are then incremented by *n*.

In other words, an invocation of this method of the form `dst.put(src)` has exactly the same effect as the loop

```
    while (src.hasRemaining())
        dst.put(src.get());
```

except that it first checks that there is sufficient space in this buffer and it is potentially much more efficient.

Parameters:

`src - The source buffer from which chars are to be read; must not be this buffer`

Returns:

`This buffer`

Throws:

`BufferOverflowException - If there is insufficient space in this buffer for the remaining chars in the source buffer`

`IllegalArgumentException - If the source buffer is this buffer`

`ReadOnlyBufferException - If this buffer is read-only`

put

```
public CharBuffer put(char[] src,
                      int offset,
                      int length)
```

Relative bulk *put* method *(optional operation)*.

This method transfers chars into this buffer from the given source array. If there are more chars to be copied from the array than remain in this buffer, that is, if `length > remaining()`, then no chars are transferred and a `BufferOverflowException` is thrown.

Otherwise, this method copies `length` chars from the given array into this buffer, starting at the given offset in the array and at the current position of this buffer. The position of this buffer is then incremented by `length`.

In other words, an invocation of this method of the form `dst.put(src, off, len)` has exactly the same effect as the loop

```
for (int i = off; i < off + len; i++)
    dst.put(a[i]);
```

except that it first checks that there is sufficient space in this buffer and it is potentially much more efficient.

Parameters:

`src - The array from which chars are to be read`

`offset - The offset within the array of the first char to be read; must be non-negative and no larger than array.length`

`length - The number of chars to be read from the given array; must be non-negative and no larger than array.length - offset`

Returns:

`This buffer`

Throws:

`BufferOverflowException - If there is insufficient space in this buffer`

`IndexOutOfBoundsException - If the preconditions on the offset and length parameters do not hold`

`ReadOnlyBufferException - If this buffer is read-only`

put

`public final CharBuffer put(char[] src)`

Relative bulk *put* method *(optional operation)*.

This method transfers the entire content of the given source char array into this buffer. An invocation of this method of the form `dst.put(a)` behaves in exactly the same way as the invocation

```
dst.put(a, 0, a.length)
```

Parameters:

`src - The source array`

Returns:

`This buffer`

Throws:

`BufferOverflowException` - If there is insufficient space in this buffer

`ReadOnlyBufferException` - If this buffer is read-only

put

```
public CharBuffer put(String src,
                      int start,
                      int end)
```

Relative bulk *put* method *(optional operation)*.

This method transfers chars from the given string into this buffer. If there are more chars to be copied from the string than remain in this buffer, that is, if `end - start > remaining()`, then no chars are transferred and a `BufferOverflowException` is thrown.

Otherwise, this method copies *n* = `end - start` chars from the given string into this buffer, starting at the given `start` index and at the current position of this buffer. The position of this buffer is then incremented by *n*.

In other words, an invocation of this method of the form `dst.put(src, start, end)` has exactly the same effect as the loop

```
for (int i = start; i < end; i++)
    dst.put(src.charAt(i));
```

except that it first checks that there is sufficient space in this buffer and it is potentially much more efficient.

Parameters:

`src` - The string from which chars are to be read

`start` - The offset within the string of the first char to be read; must be non-negative and no larger than `string.length()`

`end` - The offset within the string of the last char to be read, plus one; must be non-negative and no larger than `string.length()`

Returns:

This buffer

Throws:

`BufferOverflowException` - If there is insufficient space in this buffer

`IndexOutOfBoundsException` - If the preconditions on the start and end parameters do not hold

ReadOnlyBufferException - If this buffer is read-only

put

```
public final CharBuffer put(String src)
```

Relative bulk *put* method *(optional operation)*.

This method transfers the entire content of the given source string into this buffer. An invocation of this method of the form `dst.put(s)` behaves in exactly the same way as the invocation

```
dst.put(s, 0, s.length())
```

Parameters:

src - The source string

Returns:

This buffer

Throws:

BufferOverflowException - If there is insufficient space in this buffer

ReadOnlyBufferException - If this buffer is read-only

hasArray

```
public final boolean hasArray()
```

Tells whether or not this buffer is backed by an accessible char array.

If this method returns `true` then the `array` and `arrayOffset` methods may safely be invoked.

Specified by:

hasArray in class Buffer

Returns:

true if, and only if, this buffer is backed by an array and is not read-only

array

```
public final char[] array()
```

Returns the char array that backs this buffer *(optional operation)*.

Modifications to this buffer's content will cause the returned array's content to be modified, and vice versa.

Invoke the `hasArray` method before invoking this method in order to ensure that this buffer has an accessible backing array.

Specified by:

`array` in class `Buffer`

Returns:

The array that backs this buffer

Throws:

`ReadOnlyBufferException` - If this buffer is backed by an array but is read-only

`UnsupportedOperationException` - If this buffer is not backed by an accessible array

arrayOffset

```
public final int arrayOffset()
```

Returns the offset within this buffer's backing array of the first element of the buffer *(optional operation)*.

If this buffer is backed by an array then buffer position *p* corresponds to array index *p* + `arrayOffset()`.

Invoke the `hasArray` method before invoking this method in order to ensure that this buffer has an accessible backing array.

Specified by:

`arrayOffset` in class `Buffer`

Returns:

The offset within this buffer's array of the first element of the buffer

Throws:

`ReadOnlyBufferException` - If this buffer is backed by an array but is read-only

`UnsupportedOperationException` - If this buffer is not backed by an accessible array

compact

```
public abstract CharBuffer compact()
```

Compacts this buffer *(optional operation)*.

The chars between the buffer's current position and its limit, if any, are copied to the beginning of the buffer. That is, the char at index p = `position()` is copied to index zero, the char at index p + 1 is copied to index one, and so forth until the char at index `limit()` - 1 is copied to index n = `limit()` - 1 - p. The buffer's position is then set to $n+1$ and its limit is set to its capacity. The mark, if defined, is discarded.

The buffer's position is set to the number of chars copied, rather than to zero, so that an invocation of this method can be followed immediately by an invocation of another relative *put* method.

Returns:

`This buffer`

Throws:

`ReadOnlyBufferException` - If this buffer is read-only

isDirect

```
public abstract boolean isDirect()
```

Tells whether or not this char buffer is direct.

Specified by:

`isDirect` in class `Buffer`

Returns:

`true if, and only if, this buffer is direct`

hashCode

```
public int hashCode()
```

Returns the current hash code of this buffer.

The hash code of a char buffer depends only upon its remaining elements; that is, upon the elements from `position()` up to, and including, the element at `limit()` - 1.

Because buffer hash codes are content-dependent, it is inadvisable to use buffers as keys in hash maps or similar data structures unless it is known that their contents will not change.

Overrides:

`hashCode` in class `Object`

Returns:

`The current hash code of this buffer`

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

equals

```
public boolean equals(Object ob)
```

Tells whether or not this buffer is equal to another object.

Two char buffers are equal if, and only if,

1. They have the same element type,
2. They have the same number of remaining elements, and
3. The two sequences of remaining elements, considered independently of their starting positions, are pointwise equal.

A char buffer is not equal to any other type of object.

Overrides:

`equals` in class `Object`

Parameters:

`ob - The object to which this buffer is to be compared`

Returns:

`true if, and only if, this buffer is equal to the given object`

See Also:

`Object.hashCode()`, `HashMap`

compareTo

```
public int compareTo(CharBuffer that)
```

Compares this buffer to another.

Two char buffers are compared by comparing their sequences of remaining elements lexicographically, without regard to the starting position of each sequence within its corresponding buffer. Pairs of `char` elements are compared as if by invoking `Character.compare(char,char)`.

A char buffer is not comparable to any other type of object.

Specified by:

`compareTo` in interface `Comparable<CharBuffer>`

Parameters:

that - the object to be compared.

Returns:

A negative integer, zero, or a positive integer as this buffer is less than, equal to, or greater than the given buffer

toString

```
public String toString()
```

Returns a string containing the characters in this buffer.

The first character of the resulting string will be the character at this buffer's position, while the last character will be the character at index `limit()` - 1. Invoking this method does not change the buffer's position.

Specified by:

`toString` in interface `CharSequence`

Overrides:

`toString` in class `Object`

Returns:

The specified string

length

```
public final int length()
```

Returns the length of this character buffer.

When viewed as a character sequence, the length of a character buffer is simply the number of characters between the position (inclusive) and the limit (exclusive); that is, it is equivalent to `remaining()`.

Specified by:

`length` in interface `CharSequence`

Returns:

The length of this character buffer

charAt

```
public final char charAt(int index)
```

Reads the character at the given index relative to the current position.

Specified by:

charAt in interface CharSequence

Parameters:

index - The index of the character to be read, relative to the position; must be non-negative and smaller than remaining()

Returns:

The character at index position() + index

Throws:

IndexOutOfBoundsException - If the preconditions on index do not hold

subSequence

```
public abstract CharBuffer subSequence(int start,
                                       int end)
```

Creates a new character buffer that represents the specified subsequence of this buffer, relative to the current position.

The new buffer will share this buffer's content; that is, if the content of this buffer is mutable then modifications to one buffer will cause the other to be modified. The new buffer's capacity will be that of this buffer, its position will be `position() + start`, and its limit will be `position() + end`. The new buffer will be direct if, and only if, this buffer is direct, and it will be read-only if, and only if, this buffer is read-only.

Specified by:

subSequence in interface CharSequence

Parameters:

start - The index, relative to the current position, of the first character in the subsequence; must be non-negative and no larger than remaining()

end - The index, relative to the current position, of the character following the last character in the subsequence; must be no smaller than start and no larger than remaining()

Returns:

The new character buffer

Throws:

IndexOutOfBoundsException - If the preconditions on start and end do not hold

append

```
public CharBuffer append(CharSequence csq)
```

Appends the specified character sequence to this buffer *(optional operation)*.

An invocation of this method of the form `dst.append(csq)` behaves in exactly the same way as the invocation

```
dst.put(csq.toString())
```

Depending on the specification of `toString` for the character sequence `csq`, the entire sequence may not be appended. For instance, invoking the `toString` method of a character buffer will return a subsequence whose content depends upon the buffer's position and limit.

Specified by:

`append` in interface `Appendable`

Parameters:

`csq` - The character sequence to append. If `csq` is null, then the four characters "null" are appended to this character buffer.

Returns:

This buffer

Throws:

`BufferOverflowException` - If there is insufficient space in this buffer

`ReadOnlyBufferException` - If this buffer is read-only

Since:

1.5

append

```
public CharBuffer append(CharSequence csq,
                         int start,
                         int end)
```

Appends
a subsequence of the specified character sequence to this buffer *(optional operation)*.

An invocation of this method of the form `dst.append(csq, start, end)` when `csq` is not `null`, behaves in exactly the same way as the invocation

```
dst.put(csq.subSequence(start, end).toString())
```

Specified by:

append in interface Appendable

Parameters:

csq - The character sequence from which a subsequence will be appended. If csq is null, then characters will be appended as if csq contained the four characters "null".

start - The index of the first character in the subsequence

end - The index of the character following the last character in the subsequence

Returns:

This buffer

Throws:

BufferOverflowException - If there is insufficient space in this buffer

IndexOutOfBoundsException - If start or end are negative, start is greater than end, or end is greater than csq.length()

ReadOnlyBufferException - If this buffer is read-only

Since:

1.5

append

public CharBuffer append(char c)

Appends the specified char to this buffer *(optional operation)*.

An invocation of this method of the form `dst.append(c)` behaves in exactly the same way as the invocation

```
dst.put(c)
```

Specified by:

append in interface Appendable

Parameters:

c - The 16-bit char to append

Returns:

This buffer

Throws:

BufferOverflowException - If there is insufficient space in this buffer

ReadOnlyBufferException - If this buffer is read-only

Since:

1.5

order

```
public abstract ByteOrder order()
```

Retrieves this buffer's byte order.

The byte order of a char buffer created by allocation or by wrapping an existing `char` array is the `native order` of the underlying hardware. The byte order of a char buffer created as a view of a byte buffer is that of the byte buffer at the moment that the view is created.

Returns:

This buffer's byte order

chars

```
public IntStream chars()
```

Description copied from interface: `CharSequence`

Returns a stream of `int` zero-extending the `char` values from this sequence. Any char which maps to a surrogate code point is passed through uninterpreted.

If the sequence is mutated while the stream is being read, the result is undefined.

Specified by:

`chars` in interface `CharSequence`

Returns:

an IntStream of char values from this sequence

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.